INFORME TÉCNICO

KickoffAgent — Agente Inteligente de Formulación de Proyectos TI

Campo	Valor
Asignatura	ISY0101 — Ingeniería de Soluciones con IA
Evaluación	Parcial N°2 — Desarrollo de un Agente Funcional
Integrantes	Génesis Valdebenito
Fecha	21 de October de 2026
Repositorio	https://github.com/GenesisValdebenito/KickoffAgent

1. Resumen Ejecutivo

KickoffAgent es un agente inteligente de tipo ReAct desarrollado con el framework LangGraph de LangChain, orientado a la automatización del proceso de formulación y evaluación de proyectos TI. El agente integra un pipeline RAG (Retrieval-Augmented Generation) sobre ChromaDB, herramientas de cálculo financiero determinista (VAN, TIR, ROI, PRI) mediante numpy-financial, memoria conversacional de corto y largo plazo, y persistencia documental en disco.

2. Arquitectura del Sistema

La arquitectura de KickoffAgent sigue el patrón **agente + herramientas orquestado por grafo de estados**. Los componentes principales son:

Componente	Función
LangGraph StateGraph	Orquestación del flujo agente → herramientas → agente con aristas condicionale
MemorySaver (LangGraph)	Memoria de corto plazo (dentro de la sesión) mediante checkpointing en RAM
Memoria Persistente (JSON)	Módulo src/memory.py que serializa el historial de sesiones en disco (sessions/)
ChromaDB (RAG)	Vector store para indexación y recuperación semántica de documentos
FastEmbed (BAAI/bge-small-en-v1.5)	Modelo de embeddings ejecutado 100% localmente sin API externa
Groq / MockDemoChat	LLM principal (qwen/qwen3.8-27b vía API) con fallback determinista offline
numpy-financial	Cálculo determinista de métricas financieras (VAN, TIR, ROI, PRI)
src/tools.py	7 herramientas: consultar RAG, calcular finanzas, guardar/editar/eliminar notas,

2.1 Flujo de Ejecución

Al iniciar, el agente: (1) detecta sesiones previas y ofrece continuar la última conversación; (2) verifica la API key o activa el modo demo; (3) indexa documentos de docs/ en ChromaDB; (4) entra en el loop interactivo Q&A; con el usuario. Cada consulta pasa por el grafo LangGraph: el nodo *agent* decide si invocar herramientas o responder directamente. Si invoca herramienta, el resultado vuelve al nodo *agent* para formular la respuesta final. La sesión se auto-guarda en disco tras cada turno.

3. Herramientas del Agente (src/tools.py)

Herramienta	Descripción
consultar_documentacion_proyecto	RAG sobre ChromaDB. Recupera fragmentos relevantes de PDFs y TXTs indexados.
calcular_metricas_financieras	Calcula VAN, TIR, ROI y PRI usando numpy-financial. Entrada: inversión, flujos, tasa.
guardar_informe_kickoff	Persiste un dictamen o informe en output/ como archivo Markdown.
editar_nota_kickoff	Sobrescribe el contenido de un archivo existente en output/.
eliminar_nota_kickoff	Elimina un archivo guardado de output/.
exportar_resumen_sesion	Exporta un resumen de la conversación actual a un archivo Markdown en output/.

sumar / restar / multiplicar / dividir	Operaciones aritméticas auxiliares de apoyo al cálculo financiero.

4. Estrategias de Memoria

4.1 Memoria de Corto Plazo (MemorySaver)

LangGraph usa MemorySaver como checkpointer en RAM, identificando cada hilo de conversación por un thread_id único. Esto permite que el agente recuerde el contexto completo de la sesión en curso sin que el usuario repita datos previamente mencionados.

4.2 Memoria de Largo Plazo (Persistencia JSON)

El módulo src/memory.py serializa el historial de mensajes (usuario + agente) en archivos JSON dentro de la carpeta sessions/. Al reiniciar el agente, detecta sesiones previas y ofrece continuar la última conversación, restaurando el contexto en el grafo de LangGraph.

4.3 Memoria Vectorial (ChromaDB RAG)

Los documentos ingresados (PDF, TXT, MD) se fragmentan con RecursiveCharacterTextSplitter (chunk_size=700, overlap=120) y se indexan en ChromaDB usando embeddings BAAI/bge-small-en-v1.5 (ejecución 100% local vía FastEmbed). El retriever recupera los k=4 fragmentos más relevantes ante cada consulta, proporcionando grounding factual basado en los documentos reales.

5. Planificación y Toma de Decisiones

El agente implementa el patrón **ReAct** (Reasoning + Acting) mediante el grafo LangGraph: el nodo *agent* razona sobre la consulta y decide si invocar una herramienta o responder directamente. La función *should_continue* inspecciona los tool_calls del último mensaje; si existen, redirige al nodo *tools*; de lo contrario, termina. Este ciclo permite planificación multi-paso (ej: consultar RAG → extraer datos → calcular VAN → redactar dictamen).

6. Instrucciones de Ejecución

Instalación de dependencias:

```
pip install -r requirements.txt
```

Ejecución normal (interactiva):

```
python src/kickoff_agent.py
```

Ejecución de pruebas automatizadas:

```
python src/kickoff_agent.py --test
```

```
pytest tests/ -v
```

Cargar un documento al iniciar:

```
python src/kickoff_agent.py ruta/al/documento.pdf
```

Comandos disponibles en el modo interactivo:

Comando	Descripción
/cargar <ruta>	Indexa un nuevo documento en caliente
/documentos	Lista los documentos activos en la base vectorial
/historial	Muestra el historial de la sesión actual
/exportar	Exporta la sesión completa a un archivo Markdown
/sesiones	Ver y gestionar sesiones guardadas persistentes
/limpiar	Reinicia la memoria conversacional (nueva sesión)
/test	Ejecuta la batería de pruebas automatizadas
salir	Guarda la sesión y finaliza

7. Evidencia de Ejecución

Las capturas de pantalla de la ejecución del agente se encuentran en la carpeta **tests/** del repositorio (test_01.png a test_04-2.png) junto con el archivo tests/Evidencia_Tests.md que documenta cada prueba con su resultado esperado y obtenido. El pipeline completo fue verificado en modo demo (sin API) y en modo live con Groq API.

8. Conclusión

KickoffAgent cumple con todos los requisitos de la rúbrica de la Evaluación Parcial N°2: integra herramientas de consulta, escritura y razonamiento; implementa memoria de corto y largo plazo con recuperación semántica vectorial; aplica estrategias de planificación ReAct con grafo de estados LangGraph; y documenta claramente la arquitectura y orquestación entre componentes. El código está versionado en GitHub con CI mediante GitHub Actions y tests automatizados con pytest.